

NVC-1B: Scaling up Neural Video Coding Models

Chuanbo Tang, Xihua Sheng^{1b}, *Member, IEEE*, Li Li^{1b}, *Senior Member, IEEE*, Dong Liu^{1b}, *Senior Member, IEEE*, and Feng Wu^{1b}, *Fellow, IEEE*

Abstract—Emerging large models have achieved notable progress in the fields of natural language processing and computer vision. However, large models for neural video coding are still unexplored. In this paper, we try to explore how to build a large neural video coding model. Based on a small baseline model, we gradually scale up the model sizes of its different coding parts, including the motion encoder-decoder, motion entropy model, contextual encoder-decoder, contextual entropy model, and temporal context mining module, and analyze the influence of model sizes on video compression performance. Then, we explore using different architectures, including CNN, mixed CNN-Transformer, and Transformer architectures, to implement the neural video coding model and analyze the influence of model architectures on video compression performance. Based on our exploration results, we design the first neural video coding model having more than 1 billion parameters – NVC-1B. Experimental results show that our large model achieves a significant video compression performance improvement over recent state-of-the-art neural video compression models. With the continuous advancement in hardware and the successful on-device deployment of large models, we anticipate that our proposed large neural video coding model can bring video coding technologies to the next level.

Index Terms—Contextual coding, large model, motion coding, neural video coding, temporal context mining.

I. INTRODUCTION

THE popularity of video applications, such as short video-sharing platforms, video conferences, and streaming television, has made the amount of video data increase rapidly. The large amount of data brings high costs for video transmission and storage. Therefore, it is urgent to compress videos efficiently to reduce the video data amount.

To decrease the costs of video transmission and storage, various advanced coding technologies have been proposed, including intra/inter-frame prediction, transform, quantization, entropy coding, and loop filters. These coding technologies have led to the development of a series of video coding standards over the past decades, such as H.264/AVC [1], H.265/HEVC [2], and H.266/VVC [3]. As computing power increases, codecs for these

video coding standards become more and more complex, leading to significant improvements in video compression performance.

Although traditional video coding standards have achieved great success, their compression performance is increasing at a slower rate than the amount of video data. With the development of neural networks and GPUs, to break through the bottleneck of video compression performance growth, researchers have begun to explore end-to-end neural video coding in recent years. Existing end-to-end neural video coding (NVC) models can be roughly classified into four classes: volume coding-based [4], [5], conditional entropy modeling-based [6], [7], implicit neural representation-based [8], [9], [10], [11] [12], motion compensated prediction-based [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33], [34], [35], [36], [37], [38], [39], [40], [41] [42], [43], [44], [45]. By drawing on advanced technologies in the fields of natural language processing and computer vision, recent neural video coding models [33], [39], [46] have surpassed the reference software of H.266/VVC standard–VTM under certain coding conditions. However, progress in neural video coding has been slower than in NLP and general computer vision. Although large models are now common in language and vision, scaling neural video coding remains largely unexplored. By contrast, learned image compression [47] has already scaled to models with up to 1 billion parameters and clear gains, while comparable large video coding models have not yet been reported.

In recent years, supported by the increasing computing power of GPUs, large models for natural language and computer vision have emerged. Large models for natural language processing and computer vision have attracted much attention recently. Large language models (LLMs) and large vision models (LVMs) are commonly first built on small models, and then their model sizes are gradually scaled up. Following the scaling law [48], increasing the model sizes can bring impressive task performance gains. In terms of LLMs, their model sizes can reach hundreds to thousands of billion (B). For example, representative LLMs developed by OpenAI, including GPT-3 [49] and GPT-4 [50], contain 175B parameters and about 1800B parameters, respectively. Similarly, DeepSeek-V3 [51], an open-source multilingual LLM with up to 236B parameters, exhibits strong generalization capabilities. Their natural language understanding ability and human language generation ability are greatly improved. In terms of LVMs, their model sizes are much smaller than those of LLMs. For example, the Segment Anything Model (SAM) [52] used for image segmentation and the Depth Anything Model [53] only contain 636 million (M) parameters and 335 M parameters. Even the largest Vision Transformer (ViT) [54] models have only

Received 11 November 2025; revised 25 March 2026; accepted 24 April 2026. Date of publication 4 May 2026; date of current version 6 August 2026. This work was supported by the Natural Science Foundation of China under Grant 62021001 and Grant U25B2010. Recommended for acceptance by B. Shi. (Chuanbo Tang and Xihua Sheng contributed equally to this work.) (Corresponding authors: Dong Liu; Feng Wu.)

The authors are with the MOE Key Laboratory of Brain-Inspired Intelligent Perception and Cognition, University of Science and Technology of China, Hefei 230093, China (e-mail: cbtang@mail.ustc.edu.cn; xhsheng@mail.ustc.edu.cn; lil1@ustc.edu.cn; dongliu@ustc.edu.cn; fengwu@ustc.edu.cn).

Digital Object Identifier 10.1109/TPAMI.2026.3689998

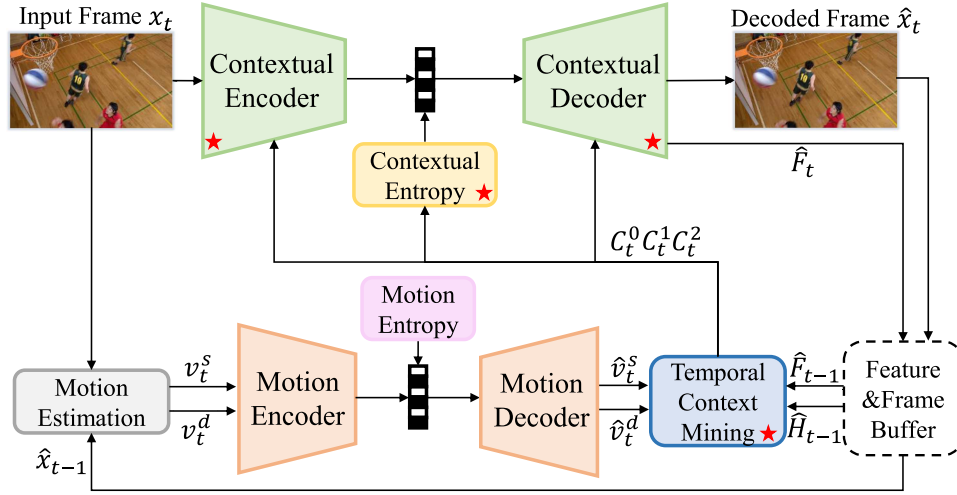


Fig. 1. Overview of our proposed large neural video coding model-NVC-1B. We explore scaling up the model size of the motion encoder-decoder, motion entropy model, contextual encoder-decoder, contextual entropy model, and temporal context mining module. Based on our exploration results, we allocate most model parameters to the modules marked with red stars.

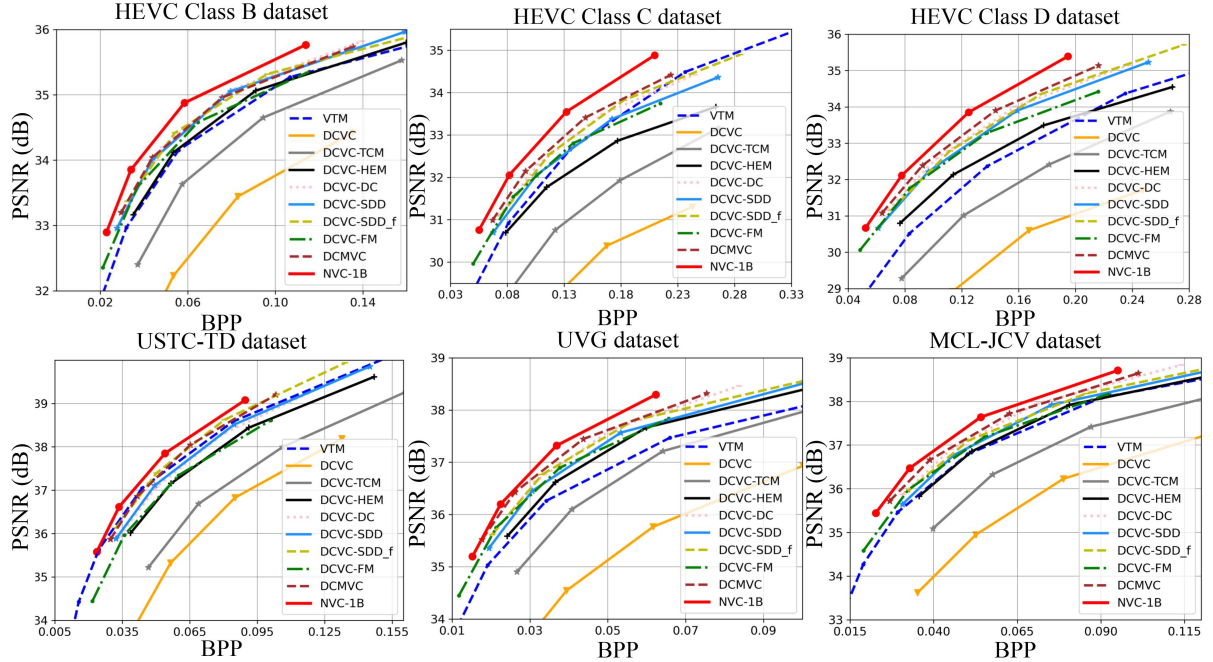


Fig. 2. Rate-distortion curves on HEVC, USTC-TD, UVG, and MCL-JCV RGB video datasets using 96-frame sequences with an intra-period of 32. Reconstruction quality is measured by RGB-PSNR.

recently grown from a few hundred million to 22B [55]. Meanwhile, recent large-scale LVMs such as FLUX.1 Kontext [56] for image generation and editing (12B), Qwen-Image [57] for image generation (20B), and Wan [58] for video generation (up to 14B) remain far smaller than typical LLMs. Although the model sizes of current LVMs are much smaller than those of LLMs, larger model sizes for vision models still greatly improve computer vision task performance [59].

Witnessing the success of LLMs and LVMs, we try to explore the effectiveness of large models for neural video coding. Considering that the training codes of most neural video coding

models are difficult to access, we adopt an open-source small model [33] as our baseline, which is based on the popular DCVC-DC architecture [39] and has a higher compression performance than the reference software of H.266/VVC—VTM-13.2 [60] under certain testing conditions. We first analyze the influence of model sizes on compression performance. Specifically, we gradually scale up the model sizes of the motion encoder-decoder, motion entropy model, contextual encoder-decoder, contextual entropy model, and temporal context mining module. Then, we analyze the influence of model architectures on compression performance. Specifically, we try to replace the CNN

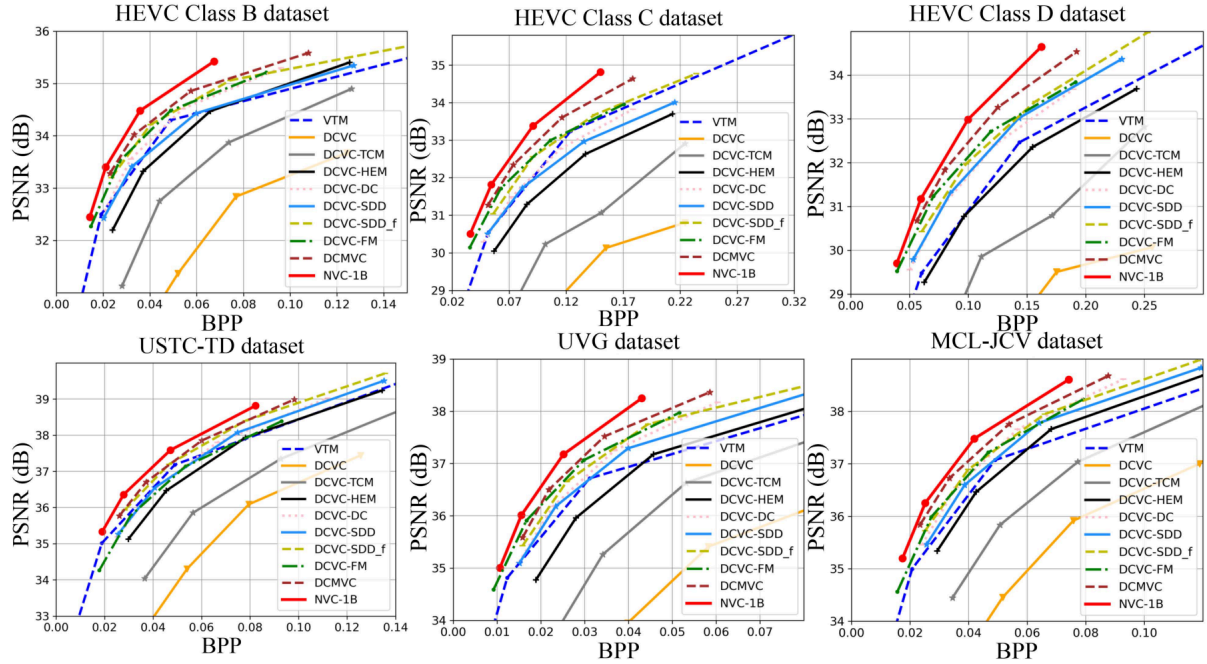


Fig. 3. Rate–distortion curves on HEVC, USTC-TD, UVG, and MCL-JCV RGB video datasets using all-frame sequences with an intra-period of -1. Reconstruction quality is measured by RGB-PSNR.

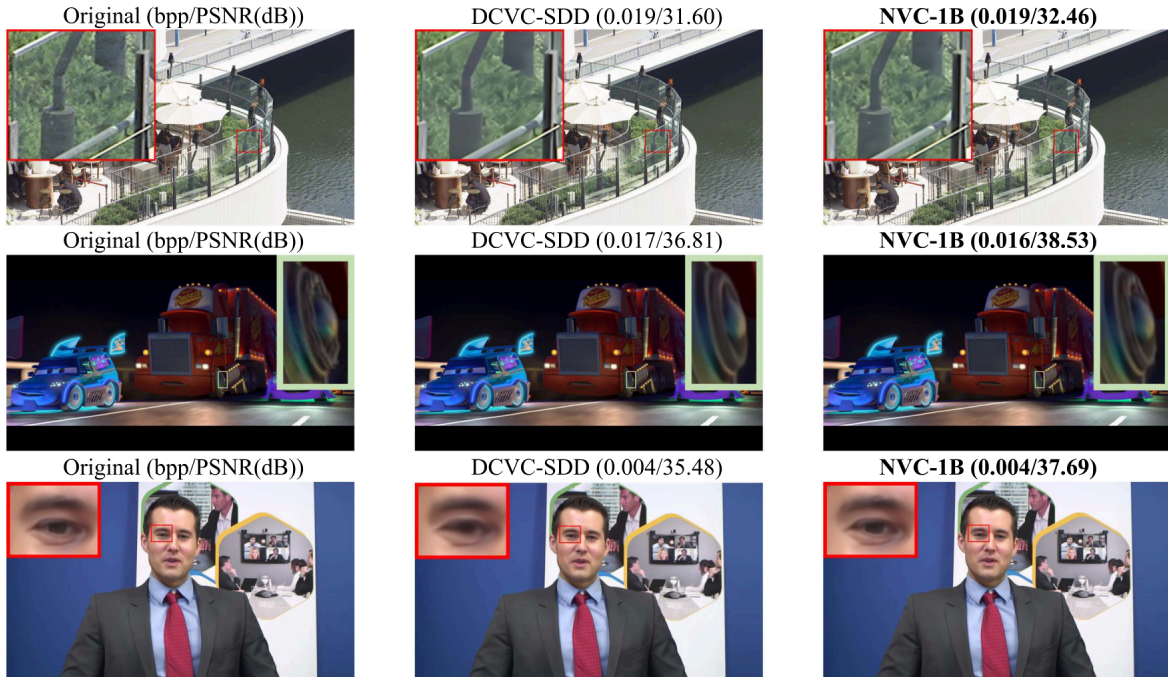


Fig. 4. Subjective quality comparison between our proposed large model—NVC-1B and its small baseline model—DCVC-SDD [33] on the 44th frame of the HEVC Class B *BQTerrace* sequence, the 8th frame of the MCL-JCV *videoSRC24* sequence, and the 48th frame of the HEVC Class E *Johnny* sequence.

architecture with the mixed CNN-Transformer or Transformer-only architecture. Finally, based on these findings, we build a large neural video coding model with 1 billion parameters—NVC-1B.

Our contributions are summarized as follows:

- We scale up the model sizes of different parts of a neural video coding model and analyze the influence of model sizes on video compression performance.

- We use different architectures to implement the neural video coding model and analyze the influence of model architectures on video compression performance.
- Based on our exploration results, we propose the first large neural video coding model with more than 1 billion parameters—NVC-1B and achieve a significant video compression performance improvement over recent state-of-the-art (SOTA) neural video compression models.

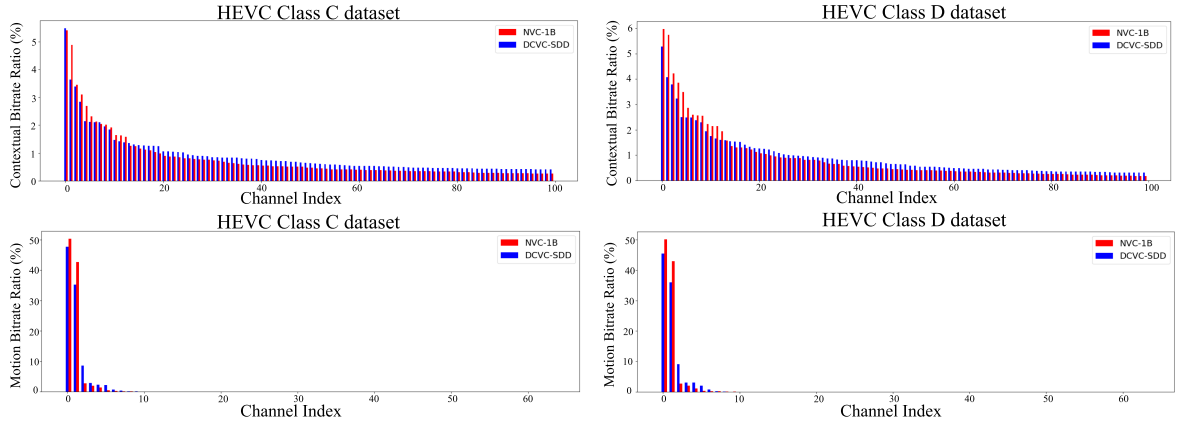


Fig. 5. Comparison of the average contextual and motion channel bitrate distribution between our proposed large model—NVC-1B and its small baseline model—DCVC-SDD on the HEVC Class C and D datasets.

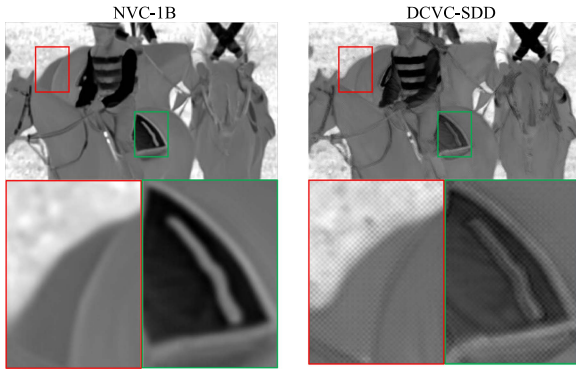


Fig. 6. Comparison of the temporal contexts predicted by our proposed large model—NVC-1B and its small baseline model—DCVC-SDD.

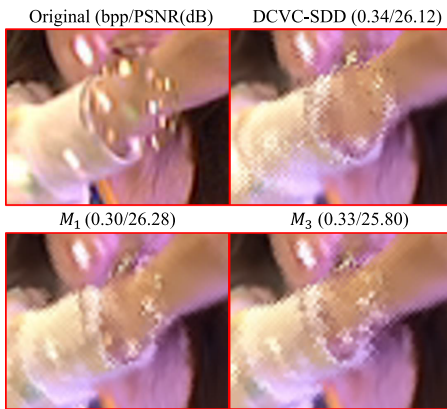


Fig. 7. Comparison of the warp frames predicted by the small baseline model—DCVC-SDD, M_1 , and M_3 models mentioned in Section IV-A1.

When video coding technologies emerged decades ago, few could predict that future computing power would be strong enough to support the deployment of complex video codecs like VTM. Nowadays, with the rapid growth of GPU/NPU compute and successful on-device deployments of large models [51], [61], [62], [63], [64], [65], [66], [67], [68], [69], we believe our exploration can help advance future video coding.

The remainder of this paper is organized as follows. Section II gives a review of related work about neural video coding, LLM, and LVM. Section III introduces the framework overview of our proposed NVC-1B. Section IV analyzes the influence of model sizes and model architectures on compression performance in detail. Section V reports the compression performance of NVC-1B and gives some analyses. Section VI gives a conclusion of this paper.

II. RELATED WORK

A. Neural Video Coding

Neural video coding [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [22], [24], [25], [25], [26], [27], [28], [29], [30], [31], [34], [37], [41], [44], [45], [46], [70], [71], [72], [73], [74], [75], [76], [77], [78], [79], [80], [81], [82] has explored a new direction for video compression in recent years. Among different kinds of coding schemes, motion compensation-based schemes have achieved state-of-the-art compression performance.

DVC [71] is a pioneering scheme of motion compensation-based neural video coding. It follows the traditional hybrid video coding framework but implements main modules with neural networks, such as motion estimation, motion compression, motion compensation, residual compression, and entropy models. Based on DVC, subsequent schemes mainly focus on how to increase the accuracy of temporal prediction. For example, Lin et al. [16] proposed M-LVC that introduces multiple reference frames for motion compensation and motion vector prediction. Agustsson et al. [18] proposed SSF that generalizes typical motion vectors to a scale-space flow for better handling complex motion. Hu et al. [17] proposed FVC that shifts temporal prediction from pixel domain to feature domain using deformable convolution [83].

Different from DVC-based schemes, DCVC [72] shifts the typical residual coding paradigm to a conditional coding paradigm. Regarding temporal prediction as a condition, DCVC feeds the condition into a contextual encoder-decoder, allowing the networks to learn how to reduce temporal redundancy automatically rather than performing explicit subtraction operations.

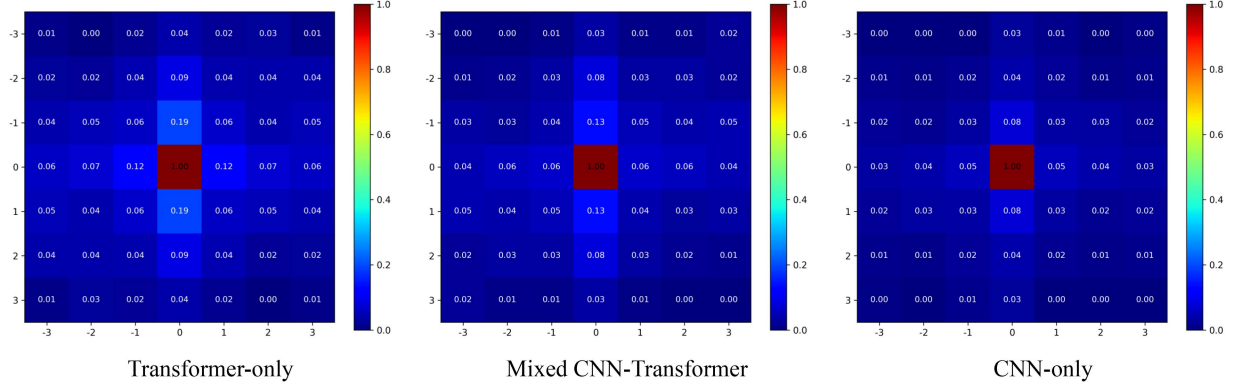


Fig. 8. Spatial correlation map of the normalized contextual latent representation $(y_t - \mu_y)/\sigma_y$ on the *BQMall_832×480* sequence at $\lambda = 85$ for models M_{24} , M_{21} , and M_{18} . Correlations are computed over relative spatial offsets in $[-10, 10]$ and then aggregated by 3×3 block averaging for better visualization. The displayed coordinates correspond to block-averaged offsets in $[-3, 3]$.

Based on DCVC, Sheng et al. [31] proposed DCVC-TCM that designs a temporal context mining module to generate multi-scale temporal contexts. This work greatly improved the compression performance of neural video coding and began to make fair comparisons with the standard reference software of traditional codecs. Following DCVC-TCM, DCVC-HEM [35], DCVC-DC [39], and DCVC-FM [46] were further proposed. They focus on introducing more efficient spatiotemporal entropy models, offset-based optical flow refinement, and hierarchical quality structure to further utilize temporal correlation. Moreover, recent works [42], [43] based on conditional coding have focused on enhancing temporal context modeling to achieve more efficient inter-frame prediction and improved compression performance. However, different from our approach, they have not explored the potential of large models to further enhance compression performance.

B. Large Language Model and Vision Model

To build a large neural video coding model, we first give a brief review of large language and vision models.

A large language model (LLM) refers to a language model containing billions of parameters with the goal of natural language understanding and human language generation. Recently, a variety of LLMs have emerged, including those developed by OpenAI (GPT-3 [49] and GPT-4 [50]), Google (GLaM [84], PaLM [85], PaLM-2 [86], and Gemini [66]), Meta (LLaMA-1 [87], LLaMA-2 [87], and LLaMA-3 [88]), as well as open-source models from other companies such as DeepSeek (DeepSeek-VL and DeepSeek-Coder [51]) and Mistral (Mistral-7B [89]). These models are usually scaled from smaller language models. Then, under the guidance of the scaling law [48], they largely scale the model sizes. For example, GPT-3 contains 175B parameters, and GPT-4 even contains about 1800B parameters. Owing to the massive scale in model sizes, LLMs have achieved significant performance across various tasks. They can better understand natural languages and generate high-quality texts based on interactive contexts, such as prompts.

The success of LLMs has led to the rise of large vision models (LVMs). Similar to LLMs, scaling up vision model sizes has

also shown improved performance for various tasks. Dehghani et al. [55] proposed Vision Transformer (ViT) [54] for image and video modeling. Its model sizes gradually increase from several hundred million to 4B [90]. Recently, the model size has even been extended to 22B [55], achieving SOTA performance across tasks ranging from few-shot classification to dense output vision tasks. Based on the ViT backbone, Cherti et al. [91] have investigated scaling laws for contrastive language-image pre-training (CLIP) [92]. The largest model based on ViT-G/14 can reach 1.8B parameters. Their investigation shows that scaling up the model size of CLIP can improve the performance of zero-shot classification of downstream vision tasks. Also based on ViT, Kirillov et al. [52] proposed a Segment Anything Model (SAM) for image segmentation. The largest SAM model has about 636 M parameters and has achieved excellent segmentation accuracy on many benchmark datasets. Yang et al. proposed a Depth Anything Model [53] with 335 M parameters for monocular image depth estimation, which demonstrates impressive depth estimation accuracy. Moreover, FLUX.1 Kontext [56] with 12B parameters delivers fast and high-fidelity in-context image generation and editing. Qwen-Image [57] with 20B parameters provides precise image generation and editing with strong text alignment and controllability. Wan [58] with a 14B configuration enables coherent and generalizable video generation with scalable configurations. Although the model sizes of current LVMs are much smaller than those of LLMs, these pioneering works of LVMs still verify the benefits of scaling up the model sizes for visual tasks. However, no research has explored the effectiveness of scaling up neural video coding models.

The expensive computational complexities bring challenges for deploying large models on resource-constrained devices. To deal with the challenge, various on-device deployment technologies were proposed for large models, such as weight quantization [61], [62], structure pruning [63], knowledge distillation [64], and low-rank factorization [65]. These advanced technologies facilitate the deployment of large models on mobile phones, such as Google Gemini Nano (7B) [66], Microsoft Phi series (3.8B) [67], and Apple OpenELM (1.1B) [68]. In addition, the continuous advancement in hardware also significantly impacts the deployment of on-device large models. For example,

Apple's Neural Engine and Huawei's Ascend AI processors have allowed on-device inference of large models [69]. Witnessing these successful cases, we believe that the on-device deployment of our large neural video coding model is feasible in the future.

III. OVERVIEW

Considering that it is difficult to access the training codes of most neural video coding models, we build the large neural video coding model—NVC-1B based on an open-source small baseline model [33], which is a model adapted from DCVC-DC [39] and has a higher compression performance than the reference software of VVC. We first give a brief overview of the framework of our NVC-1B as shown in Fig. 1

1) *Motion Estimation*: Similar to our small baseline model DCVC-SDD, we use the structure and detail decomposition-based motion estimation method to estimate the motion vectors between adjacent video frames [33]. Specifically, we first decompose the current frame x_t and the reference frame $\hat{x}_{t-1}$ into structure components ($x_t^s, \hat{x}_{t-1}^s$) and detail components ($x_t^d, \hat{x}_{t-1}^d$). Then, we estimate the motion vectors (v_t^s, v_t^d) of the structure and detail components, respectively, using a pre-trained SpyNet [93].

2) *Motion Encoder-Decoder*: We use an autoencoder-based motion encoder-decoder to compress and reconstruct the estimated motion vectors (MVs) jointly, as shown in Fig. 1. Specifically, we first perform channel-wise concatenation to the input motion vectors v_t^s and v_t^d . Then, the MV encoder compresses them into a compact latent representation m_t with the size of $H/16 \times W/16 \times C_m$. H is the height and W is the width of the input motion maps. C_m is the number of channels of the compact latent representation m_t . After quantization, the quantized latent representation $\hat{m}_t$ is signaled into a bitstream using the arithmetic encoder. After receiving the transmitted bitstream, the arithmetic decoder reconstructs the bitstream back to the quantized latent representation $\hat{m}_t$. The MV decoder then decompresses $\hat{m}_t$ back to the reconstructed motion vectors $\hat{v}_t^s$ and $\hat{v}_t^d$.

3) *Temporal Context Mining*: The temporal context mining module is essential for conditional-based neural video coding schemes [31], [33], [34], [35], [37], [39], [46], [72]. Given a reference feature $\hat{F}_t$, the temporal context mining module first uses a feature pyramid to extract multi-scale features from $\hat{F}_t$. Then, it uses multi-scale motion vectors to perform feature-based motion compensation to these features and learn multi-scale temporal contexts $\tilde{C}_t^0, \tilde{C}_t^1, \tilde{C}_t^2$. To handle motion occlusion, following our baseline [33], we use a ConvLSTM-based long-term reference generator to accumulate the historical information of each reference feature and learn a long-term reference feature $\hat{F}_{t-1}$. Then, we fuse $\hat{F}_{t-1}$ with $\tilde{C}_t^0, \tilde{C}_t^1, \tilde{C}_t^2$ to generate long short-term fused temporal contexts C_t^0, C_t^1, C_t^2 .

4) *Contextual Encoder-Decoder*: We use an autoencoder-based contextual encoder and decoder to compress and reconstruct the input frame x_t . The contextual encoder compresses x_t into a compact latent representation y_t with the size of $H/16 \times W/16 \times C_y$. C_y is the number of channels of the compact latent representation y_t . In the encoding procedure,

multi-scale temporal contexts C_t^0, C_t^1, C_t^2 learned by the temporal context mining module are channel-wise concatenated into the contextual encoder to reduce temporal redundancy. Then, quantization is performed on y_t , and the arithmetic encoder converts the quantized latent representation $\hat{y}_t$ to a bitstream. After receiving the transmitted bitstream, the arithmetic decoder reconstructs it to $\hat{y}_t$. The contextual decoder decompresses $\hat{y}_t$ to a reconstructed frame $\hat{x}_t$. In the decoding procedure, the multi-scale temporal contexts C_t^0, C_t^1, C_t^2 are also concatenated into the contextual decoder. Before obtaining $\hat{x}_t$, an intermediate feature $\hat{F}_t$ of the contextual decoder with the size of $H \times W \times C_F$ is regarded as the reference feature for encoding/decoding the next frame.

5) *Entropy Model*: We use the factorized entropy model [94] for hyperprior and the Laplace distribution [95] to model the motion and contextual compact latent representations $\hat{m}_t$ and $\hat{y}_t$. When estimating the mean and scale of the Laplace distribution of $\hat{m}_t$ and $\hat{y}_t$, we combine the hyperprior, latent prior, and the spatial prior generated by the quadtree partition-based spatial entropy model [33], [39] together. For $\hat{y}_t$, we also introduce a temporal prior learned from the smallest-resolution temporal context C_t^2 .

IV. METHODOLOGY

To explore the influence of model size on compression performance, we gradually scale up the model sizes of different parts of our small neural video coding model [33], including its motion encoder-decoder, motion entropy model, contextual encoder-decoder, contextual entropy model, and temporal context mining module. Since most existing neural video coding models use a pre-trained optical flow model for motion estimation and focus on designing other coding parts, we do not scale up the model size of the motion estimation module in this work. To explore the influence of model architecture on compression performance, we use different architectures, such as mixed CNN-Transformer and Transformer architectures, to implement the neural video coding model. In the exploration procedure, we regard our small baseline model—DCVC-SDD (21 M parameters) without multiple-frame cascaded finetune as the anchor to reduce training time.

A. Influence of Model Size

1) *Scaling up for Motion Encoder-Decoder*: We increase the number of intermediate feature channels and insert more residual blocks to scale up the model size of the motion encoder and decoder. We build three models (M_1, M_2, M_3) with 52 M, 71 M, and 96 M parameters, respectively. As illustrated in Table I the comparison results indicate that a slight increase in the model size of the motion encoder and decoder results in performance gains. However, continuously increasing the model size of the motion encoder-decoder will reduce the performance gain. For example, when the model size of the motion encoder-decoder is increased to 52 M, an average performance gain of 11.9% can be achieved on HEVC Class B, C, D, and E datasets. However, if we further increase its model size to 96 M parameters, the model instead suffers a 0.9% performance loss. In

TABLE I

EFFECTIVENESS OF SCALING UP THE MODEL SIZE OF THE MOTION ENCODER-DECODER (M_1, M_2, M_3) AND MOTION ENTROPY MODEL (M_4, M_5, M_6). BD-RATE (%) FOR RGB-PSNR IS EVALUATED AGAINST OUR SMALL 21M-PARAMETER BASELINE MODEL [33].

	Params	Complexity (kMACs/pixel)	B	C	D	E	Avg
M_1	52M	4580	-6.4	-15.4	-16.4	-9.3	-11.9
M_2	71M	5988	-0.2	-2.4	-1.5	-1.5	-1.4
M_3	96M	9721	5.6	-1.5	-5.2	4.5	0.9
M_4	128M	5062	-6.9	-16.0	-16.7	-10.9	-12.6
M_5	177M	6855	-2.7	-3.8	-1.9	-7.1	-3.9
M_6	226M	11310	3.7	-3.9	-7.1	2.5	-1.2

addition, we observe that larger motion encoder-decoders make the training processes unstable. Training crashes occur more frequently, even though we reduce the learning rates or increase the training epochs. These exploration results indicate that the motion encoder-decoder is not always the larger the better. More analysis can be found in Sections V-C2 and V-C4.

2) *Scaling up for Motion Entropy Model*: Based on the abovementioned M_1, M_2, M_3 models, we continue to increase the channel number of the motion latent representation $\hat{m}_t$ and associated hyperprior $\hat{z}_t^m$. Besides, we also scale up the model size of motion hyper-encoder, hyper-decoder, and quadtree partition-based spatial context models [33], [39] by increasing the number of intermediate feature channels. We build three models (M_4, M_5, M_6) with 128 M, 177 M, and 226 M parameters, respectively. As illustrated in Table I, the comparison results indicate that the increase in the model size of the motion entropy model can bring a little compression performance gain. For example, based on the M_1 model with 52M parameters, the M_4 model increases the average performance gain from 11.9% to 12.6% on the HEVC Class B, C, D, and E datasets. This is mainly because a larger motion entropy model can improve the entropy modeling of the motion latent representation. However, even though a larger motion entropy model can improve compression performance, it cannot compensate for the performance loss caused by a larger motion encoder-decoder. Table I also reports the complexity variation. It should be noted that the increase in parameter count and the increase in computational complexity are coupled but not strictly linear. For example, although M_3 and M_4 have relatively similar parameter scales, their complexities differ significantly, because M_3 mainly scales up the motion encoder-decoder operating on higher-resolution features, while M_4 is built on M_1 by further scaling the motion entropy model, whose computation is largely performed on lower-resolution latent representations.

3) *Scaling up for Contextual Encoder-Decoder*: Based on our small baseline model with 21 M parameters, we increase the number of intermediate feature channels and insert more residual blocks to scale up the model size of the contextual encoder and decoder. We build three models (M_7, M_8, M_9) with 50 M, 77 M, and 90 M parameters, respectively. As presented in Table II, increasing the model size of the context encoder and decoder can effectively improve compression performance. Different from the motion encoder and decoder, further increasing the model size of the contextual encoder and decoder can result in

TABLE II

EFFECTIVENESS OF SCALING UP THE MODEL SIZE OF CONTEXTUAL ENCODER-DECODER (M_1, M_2, M_3) AND CONTEXTUAL ENTROPY MODEL (M_4, M_5, M_6). BD-RATE (%) FOR RGB-PSNR IS EVALUATED AGAINST OUR SMALL 21M-PARAMETER BASELINE MODEL [33].

	Params	Complexity (kMACs/pixel)	B	C	D	E	Avg
M_7	50M	5199	-5.7	-10.6	-11.7	-9.7	-9.4
M_8	77M	7549	-6.0	-14.2	-13.6	-11.2	-11.3
M_9	90M	8142	-6.4	-14.8	-13.8	-12.6	-11.9
M_{10}	92M	5427	-7.7	-13.6	-11.9	-11.9	-11.3
M_{11}	199M	8218	-8.9	-14.6	-12.8	-13.6	-12.4
M_{12}	212M	9811	-9.4	-15.5	-14.1	-14.2	-13.3

sustained and stable performance gains. For example, when the model size of the contextual encoder and decoder is increased to 50 M, an average performance gain of 9.4% can be achieved on HEVC Class B, C, D, and E datasets. When the model size of the contextual encoder and decoder is further increased to 90 M, the performance gain can reach an average improvement of 11.9%. The results show that a larger contextual encoder-decoder can improve the transform capability to capture more compact latent representations of video textures. More analysis can be found in Section V-C2.

4) *Scaling up for Contextual Entropy Model*: Based on M_7, M_8, M_9 models, we continue to scale up the model size of their contextual entropy models. We increase the channel number of the contextual latent representation $\hat{y}_t$ and associated hyperprior $\hat{z}_t^y$. In addition, we increase the channel number of intermediate features of the contextual entropy model, including the contextual hyper-encoder, hyper-decoder, and quadtree partition-based spatial context models [33], [39] (contextual spatial prior adaptor blocks and contextual spatial prior fusion blocks). We build three models (M_{10}, M_{11}, M_{12}) with 92 M, 199 M, and 212 M parameters, respectively. As presented in Table II, increasing the model size of the context encoder and decoder can bring stable compression performance improvement. For example, when the model size of M_9 model is increased from 90 M to 212 M by scaling up its contextual entropy model, an additional on average performance gain of 1.4% performance gain can be achieved on HEVC Class B, C, D, and E datasets. The results show that a larger contextual entropy model can improve the entropy modeling of the contextual latent representation.

5) *Scaling up for Temporal Context Mining*: Temporal context mining is an intermediate link between the motion encoder-decoder and the contextual encoder-decoder. To explore the influence of its model size, we increase the number of intermediate feature channels N and insert more residual blocks to scale up its model size. Based on our small baseline model with 21M parameters, we build three models (M_{13}, M_{14}, M_{15}) with 26 M, 47 M, and 75 M parameters, respectively. As shown in Table III, scaling up the model size of the temporal context mining module can improve compression performance stably. For example, when the model size is increased from 26 M to 75 M, an average performance gain can be increased from 3.7% to 10.0% on the HEVC Class B, C, D, and E datasets. To explore whether the gain of scaling up the temporal context mining module can be superimposed with the gain of scaling up the contextual

TABLE III

EFFECTIVENESS OF SCALING UP THE MODEL SIZE OF SINGLE TEMPORAL CONTEXT MINING MODULE (M_{13} , M_{14} , M_{15}) AND THE COMBINED SCALING OF BOTH THE CONTEXTUAL ENCODER-DECODER AND CONTEXTUAL ENTROPY MODEL (M_{16} , M_{17} , M_{18}). BD-RATE (%) FOR RGB-PSNR IS EVALUATED AGAINST OUR SMALL 21M-PARAMETER BASELINE MODEL [33].

	Params	Complexity (kMACs/pixel)	B	C	D	E	Avg
M_{13}	26M	3413	-2.3	-4.5	-5.1	-3.0	-3.7
M_{14}	47M	5324	-4.1	-5.1	-8.6	-7.8	-6.4
M_{15}	75M	7946	-6.7	-7.9	-12.0	-13.5	-10.0
M_{16}	144M	10228	-9.1	-16.2	-16.8	-10.1	-13.1
M_{17}	252M	20095	-11.4	-17.1	-19.1	-14.5	-15.5
M_{18}	265M	20720	-14.7	-19.8	-21.2	-17.5	-18.3

encoder-decoder and contextual entropy model, based on M_{10} , M_{11} , M_{12} , we build another three models (M_{16} , M_{17} , M_{18}) with 144 M, 252 M, and 265 M parameters, respectively. As shown in Table III, based on the model with a larger contextual encoder-decoder and contextual entropy model, scaling up the model size of the temporal context mining module can bring additional performance gain. For example, based on M_{12} with 212M parameters, increasing its temporal context mining module to build the model M_{18} with 265 M parameters increases the average performance gain from 13.3% to 18.3% across the HEVC Class B, C, D, and E datasets. The results show that a larger temporal context mining module can help make full use of the temporal correlation. More analysis can be found in Section V-C3.

B. Influence of Model Architecture

In addition to exploring the influence of model size, we also explore the influence of model architecture. In Section IV-A, all the models adopt CNN architectures. Considering Transformer has shown impressive performance gain in [7], [76], [78], [97], [98], in this section, we try to replace the CNN architecture with mixed CNN-Transformer or Transformer architectures. Since the experimental results presented in Section IV-A indicate that scaling up the model sizes of contextual encoder-decoder, contextual entropy model, and temporal context mining module can bring stable compression performance improvement, we try new model architectures on these modules that have been proven to work.

1) *Mixed CNN-Transformer Architecture*: In terms of the mixed CNN-Transformer architecture, we build three models M_{19} , M_{20} , M_{21} . For model M_{19} , we first try to replace the residual block between the convolution layers of the contextual encoder and decoder with the parallel CNN-Transformer block [96], but we find that this block causes instability in the training process. If we only maintain their window-based Transformer branch, the instability can be solved. Therefore, we only use the window-based Transformer layers to replace the residual blocks. Then, we scale up its model size to 42M to compare it with its CNN-architecture counterpart— M_7 model (50M). For model M_{20} , based on M_{19} , we continue to insert the same Transformer layers into the CNN-based entropy model to build a contextual entropy model with mixed CNN-Transformer architecture. We scale up its model size to 240M to compare it with its CNN architecture counterpart— M_{12} model (212M). For

TABLE IV

EFFECTIVENESS OF MIXED CNN-TRANSFORMER ARCHITECTURE (M_{19} , M_{20} , M_{21}). THE TRANSFORMER BRANCHES OF [96] ARE INSERTED INTO THE CONTEXTUAL ENCODER-DECODER, CONTEXTUAL ENTROPY MODEL, AND TEMPORAL CONTEXT MINING MODULE FOR COMPARING THE CNN MODELS (M_7 , M_{12} , M_{18}). BD-RATE (%) FOR RGB-PSNR IS EVALUATED AGAINST OUR SMALL 21M-PARAMETER BASELINE MODEL [33].

	Params	Complexity (kMACs/pixel)	B	C	D	E	Avg
M_7	50M	5199	-5.7	-10.6	-11.7	-9.7	-9.4
M_{12}	212M	9811	-9.4	-15.5	-14.1	-14.2	-13.3
M_{18}	265M	20720	-14.7	-19.8	-21.2	-17.5	-18.3
M_{19}	42M	4973	-3.4	-6.7	-4.3	-3.8	-4.6
M_{20}	240M	9887	-7.0	-9.3	-5.5	-7.0	-7.2
M_{21}	267M	22481	-2.0	-2.8	2.0	0.5	-0.6

model M_{21} , based on M_{20} , we insert the same window-based Transformer layers after each residual block of each level of the temporal context mining module. We scale up its model size to 267M to compare it with its CNN architecture counterpart— M_{18} model (265 M).

As illustrated in Table IV, for models M_{19} and M_{20} , inserting the Transformer blocks into the contextual encoder-decoder and contextual entropy model can bring performance gain over the small baseline model. This is mainly because the window-based Transformer allows to utilize global correlation to further reduce spatial and temporal redundancy. This phenomenon is consistent with existing Transformer-enhanced small video coding models [7], [76], [78], [97], [98]. However, when compared with their CNN-architecture counterparts of similar model sizes, CNN-architecture counterparts can obtain higher video coding performance gain. For example, comparing M_{20} and M_{12} , which both have larger contextual encoder-decoder and larger contextual entropy model, the average performance gain of M_{12} is 13.3% but that of M_{20} is only 7.2% on the HEVC Class B, C, D, and E datasets. This may be because increasing the depth of CNN-based networks gains a larger receptive field than window-based Transformer layers, thus improving global feature capabilities.

For model M_{21} , inserting window-based Transformer layers into the temporal context mining module brings performance loss. The average compression performance gain drops from 7.2% achieved by M_{20} to 0.6% across the HEVC Class B, C, D, and E datasets. The results indicate that Transformer layers may not be suitable for the hierarchical structure of the temporal context mining module. This may be the reason why [99] only inserted Transformer layers into the lowest level of the temporal context mining module to learn the smallest scale temporal context.

2) *Transformer-Only Architecture*: In terms of the Transformer-only architecture, we build three models M_{22} , M_{23} , M_{24} based on our small baseline model. For model M_{22} , we replace all the convolutional layers and residual blocks of the contextual encoder and decoder with the window-based Transformer branches of [96] to build a contextual encoder-decoder with Transformer architecture. We scale up its model size to 44M to compare it with its CNN-architecture counterpart— M_7 model (50M). For model M_{23} , based on M_{22} , we further replace the convolutional layers and depth blocks in the contextual entropy model with the same

TABLE V

EFFECTIVENESS OF TRANSFORMER ARCHITECTURE (M_{22} , M_{23} , M_{24}). THE TRANSFORMER BRANCHES OF [96] IS REPLACED IN THE CONTEXTUAL ENCODER-DECODER, CONTEXTUAL ENTROPY MODEL, AND TEMPORAL CONTEXT MINING MODULE FOR COMPARING THE CNN MODELS (M_7 , M_{12} , M_{18}). BD-RATE (%) FOR RGB-PSNR IS EVALUATED AGAINST OUR SMALL 21M-PARAMETER BASELINE MODEL [33].

	Params	Complexity (kMACs/pixel)	B	C	D	E	Avg
M_7	50M	5199	-5.7	-10.6	-11.7	-9.7	-9.4
M_{12}	212M	9811	-9.4	-15.5	-14.1	-14.2	-13.3
M_{18}	265M	20720	-14.7	-19.8	-21.2	-17.5	-18.3
M_{22}	50M	5734	-4.3	-1.1	-6.0	-5.1	-4.1
M_{23}	212M	11018	-4.5	-4.9	-6.9	-4.2	-5.1
M_{24}	265M	21921	30.1	61.7	41.5	48.3	45.4

Transformer layers to build a contextual entropy model with Transformer architecture. We scale up its model size to 208M to compare it with its CNN architecture counterpart— M_{12} model (212M). For model M_{24} , based on M_{23} , we replace all the convolutional layers and residual blocks of the temporal context mining module with the same window-based Transformer layers. We scale up its model size to 232M to compare it with its CNN architecture counterpart— M_{18} model (265 M).

As illustrated in Table V, for model M_{22} , the Transformer-based contextual encoder-decoder can bring a little performance gain over the small baseline model. However, when compared with its CNN architecture counterpart M_7 with similar model sizes, its performance gain is much smaller. Compared with model M_{22} , model M_{23} achieves an average performance gain of 1.0% across the HEVC B, C, D, and E datasets, but it requires a more elaborate training strategy and a longer training process. However, its performance gain is also much smaller than its CNN architecture counterpart— M_{12} model (265M). For model M_{24} , even with a longer training process, the training of the model crashes, resulting in an average performance drop of 45.4% compared to the small baseline model. The main reason is that the Transformer-only architecture makes the training process unstable, which indicates that convolutional layers are necessary for our model.

C. Summary of Exploration Results

Based on the abovementioned exploration results, we make the following summary:

- Slightly scaling up the model size of the motion encoder-decoder and motion entropy model brings compression performance gains, while further increasing the model size can lead to performance degradation.
- Scaling up the model sizes of the contextual encoder-decoder, contextual entropy model, and temporal context mining module can bring continuous compression performance improvement.
- Inserting window-based Transformer layers or more residual blocks into our CNN-based models can both obtain better global feature extraction ability to improve the compression performance.
- With a larger model size, the training stability and compression performance of our models with CNN architecture

TABLE VI

NUMBER OF PARAMETERS FOR EACH CODING MODULE OF OUR PROPOSED NVC-1B

	Number of Parameters
Motion Estimation	0.96M
Motion Encoder-Decoder	0.81M
Motion Entropy Model	2.50M
Contextual Encoder-Decoder	504.59M
Contextual Entropy Model	435.88M
Temporal Context Mining	411.21M
Totally	1355.95M = 1.36B

TABLE VII

TRAINING STRATEGY OF OUR MODEL FOR ENCODING RGB VIDEOS WHEN THE DISTORTION IS MEASURED BY RGB PSNR

Frames	Network	Loss	Learning Rate	Epoch
2	Inter	L_t^{meD}	$1e-4$	2
2	Inter	L_t^{meRD}	$1e-4$	6
2	Recon	L_t^{recD}	$5e-5$	6
3	Inter	L_t^{meRD}	$1e-4$	2
3	Recon	L_t^{recD}	$5e-5$	3
4	Recon	L_t^{recD}	$5e-5$	3
6	Recon	L_t^{recD}	$5e-5$	3
2	Recon	L_t^{recRD}	$5e-5$	6
3	Recon	L_t^{recRD}	$5e-5$	3
4	Recon	L_t^{recRD}	$5e-5$	3
6	Recon	L_t^{recRD}	$5e-5$	3
2	All	L_t^{all}	$5e-5$	15
3	All	L_t^{all}	$5e-5$	15
4	All	L_t^{all}	$5e-5$	15
6	All	L_t^{all}	$5e-5$	10
6	All	L_t^{all}	$1e-5$	10
6	All	L_t^{all}	$5e-6$	5
6	All	L_T^{all}	$1e-5$	2
6	All	L_T^{all}	$1e-6$	2
6	All	L_T^{all}	$5e-7$	2
6	All	L_T^{all}	$1e-7$	4
32	All	L_T^{all}	$1e-5$	2
32	All	L_T^{all}	$5e-6$	4
32	All	L_T^{all}	$1e-6$	6

are better than those with Transformer-only and Mixed CNN-Transformer architecture.

According to the summary of exploration results, we build a large neural video coding model—NVC-1B with a CNN architecture. Under the limited GPU memory condition, and in order to ensure the stability of training, we allocate most of the model parameters to the contextual encoder-decoder, contextual entropy model, and temporal context mining module. The number of parameters for each coding module of our proposed NVC-1B is listed in Table VI.

D. Model Training

We design an elaborate training strategy for our proposed NVC-1B model, ensuring that each of its modules is fully trained. Table VII lists the detailed training stages. The training stages can be classified into 6 classes according to the training loss functions: L_t^{meD} , L_t^{meRD} , L_t^{recD} , L_t^{recRD} , L_t^{all} , and L_T^{all} . Among them, when using L_t^{meD} and L_t^{meRD} as the loss functions, we only train the motion parts (Inter). When using L_t^{recD}

and L_t^{recRD} as the loss functions, we only train the temporal context mining and contextual parts (Rec). When using L_t^{all} or L_T^{all} as the loss function, we train all parts of the model (All).

- As described in (1), L_t^{meD} calculates the distortion D_t^m between x_t and its warping frame $\tilde{x}_t$, which is used to obtain the high-fidelity reconstructed motion vectors.

$$L_t^{meD} = w_t \cdot \lambda \cdot D_t^m. \quad (1)$$

- As described in (2), based on L_t^{meD} , L_t^{meRD} takes the trade-off between the fidelity D_t^m and the consumed bitrate R_t^m of motion vectors into account. R_t^m denotes the joint bitrate used for encoding the quantized motion latent representation $\hat{n}_t$ and its associated hyperprior.

$$L_t^{meRD} = w_t \cdot \lambda \cdot D_t^m + R_t^m. \quad (2)$$

- As described in (3), L_{recD} calculates the distortion D_t^y between x_t and its reconstructed frame $\hat{x}_t$, which is used to generate a high-quality reconstructed frame.

$$L_t^{recD} = w_t \cdot \lambda \cdot D_t^y. \quad (3)$$

- As described in (4), based on L_t^{recD} , L_t^{recRD} takes the trade-off between the quality of reconstructed frame $\hat{x}_t$ and the consumed bitrate R_t^y of contextual latent representation $\hat{y}_t$. R_t^y denotes the joint bitrate used for encoding the quantized contextual latent representation $\hat{y}_t$ and its associated hyperprior.

$$L_t^{recRD} = w_t \cdot \lambda \cdot D_t^y + R_t^y. \quad (4)$$

- As described in (5), L_t^{all} takes the trade-off between the quality of the reconstructed frame $\hat{x}_t$ and all the consumed bitrate of the coded frame into account.

$$L_t^{all} = w_t \cdot \lambda \cdot D_t^y + R_t^m + R_t^y. \quad (5)$$

- As described in (6), based on L_{all} , L_T^{all} calculates the average loss of multiple frames, which is to achieve a cascaded fine-tuning for reducing the error propagation [31], [33], [35], [39].

$$\begin{aligned} L_T^{all} &= \frac{1}{T} \sum_t L_t^{all} \\ &= \frac{1}{T} \sum_t \{w_t \cdot \lambda \cdot D_t^y + R_t^m + R_t^y\}. \end{aligned} \quad (6)$$

We use the Lagrangian multiplier λ to control the trade-off between the bitrate and distortion. Following [33], [39], we add a periodically varying weight w_t for each P-frame before the Lagrangian multiplier λ .

Compared with the training strategy of our small baseline model [33], we use smaller learning rates because we found that original learning rates would bring training gradient explosion, which is similar to [49]. In addition, since more parameters are allocated to the contextual encoder-decoder, contextual entropy model, and temporal context mining module, these modules need to be trained longer to make the corresponding loss functions converge. Therefore, we have to increase the training epochs of these modules. Moreover, to ensure a fair

comparison with recent SOTA neural video codecs [42], [43], [46] and to strengthen long-chain coding adaptation, we adopt long-sequence training with 32 frames to optimize our model.

V. EXPERIMENTS

A. Experimental Setup

1) *Training and Testing Data*: In this work, for training, we still use the videos of the Vimeo-90 k dataset. Following most existing neural video coding models [31], [33], [35], [72], we randomly crop the original videos into 256×256 patches for data augmentation. Besides, we follow [46] to utilize the raw Vimeo videos¹ to collect a subset of 9000 sequences for 32-frame cascaded training. For testing, we use HEVC dataset [100], USTC-TD dataset [101], UVG dataset [102], and MCL-JCV dataset [103] in RGB format. These datasets contain videos with different contents, motion patterns, and resolutions, which are commonly used to evaluate the performance of neural video coding models. When testing RGB videos, we convert the videos in YUV420 format to RGB format using FFmpeg.

2) *Implementation Details*: **For training**, following [33], [39], we set 4 base λ values (85, 170, 380, 840) to control the rate-distortion trade-off. For hierarchical quality, we set the periodically varying weight w_t before λ as (0.5, 1.2, 0.5, 0.9). We implement our NVC-1B model with PyTorch. AdamW [104] is used as the optimizer, and batch size is set to 32. Before the multi-frame cascaded fine-tuning stage, we train the NVC-1B model on 32 NVIDIA Ampere Tesla A40 (48 G memory) GPUs for 85 days. In the multi-frame cascaded fine-tuning stage, we train the NVC-1B model on 4 NVIDIA A800 PCIe (80 G memory) GPUs for 68 days. To alleviate the CUDA memory pressure induced by 32-frame cascaded training, partial cascaded finetuning strategy [43] and Forward Recomputation Backpropagation (FRB)² are used. The 32 frames are divided into five groups, each with the number of frames: {1, 8, 8, 8, 7}.

For inference, we adopt tiled processing for 1080p sequences under CUDA memory constraints, similar to the block-wise processing in traditional codecs, where each frame is split into two 960×1080 regions and processed sequentially.

3) *Test Configurations*: As with most previous neural video coding models, we focus on the low-delay coding scenario in this paper. Following [46], we test each video sequence with 96 frames, setting the intra-period to 32, and also test with the intra-period set to -1 for all frames. For traditional codecs, we choose VTM-13.2 [60] and ECM-13.0 [105] as our benchmarks. VTM is the official reference software of H.266/VVC, and ECM is the prototype of the next-generation traditional codec. We use *encoder_lowdelay_vtm* and *encoder_lowdelay_ecm* configurations for VTM and ECM, respectively. The detailed commands are shown as follows.

¹<http://toflow.csail.mit.edu>

²<https://qywu.github.io/2019/05/22/explore-gradient-checkpointing.html>

TABLE VIII
BD-RATE (%) COMPARISON FOR RGB-PSNR. 96 FRAMES WITH INTRA-PERIOD=32. THE ANCHOR IS VTM.

	UVG	MCL-JCV	HEVC B	HEVC C	HEVC D	HEVC E	USTC-TD	Average
VTM	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
DCVC	133.9	106.6	119.6	152.5	110.9	274.8	139.6	148.3
DCVC-TCM	23.1	38.2	32.8	62.1	29.0	75.8	75.3	48.0
DCVC-HEM	-17.2	-1.6	-0.7	16.1	-7.1	20.7	20.6	4.4
DCVC-DC	-25.9	-14.4	-13.9	-8.8	-27.7	-19.1	10.8	-14.1
DCVC-SDD	-19.7	-7.1	-13.7	-2.3	-24.9	-8.4	7.7	-9.8
DCVC-SDD_f	-25.6	-10.3	-12.9	-7.6	-25.9	-23.6	-2.7	-15.5
DCVC-FM	-20.4	-8.1	-10.3	-8.4	-25.8	-21.9	25.7	-9.9
DCMVC	-30.6	-17.3	-14.5	-14.4	-31.6	-28.1	0.4	-19.4
Ours	-35.4	-24.8	-28.4	-26.2	-39.1	-31.0	-10.0	-27.8

DCVC-SDD [33] is our small baseline model.

DCVC-SDD_f denotes DCVC-SDD trained with 32-frame cascaded training.

The quality indexes of DCVC-FM are set as 36, 45, 54, 63 to match the bit-rate range of DCVC-DC.

TABLE IX
BD-RATE (%) COMPARISON FOR RGB-PSNR. ALL FRAMES WITH INTRA-PERIOD=-1. THE ANCHOR IS VTM.

	UVG	MCL-JCV	HEVC B	HEVC C	HEVC D	HEVC E	USTC-TD	Average
VTM	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
DCVC	314.1	185.1	291.7	266.9	224.6	2083.2	221.2	512.4
DCVC-TCM	109.8	69.9	106.5	132.2	81.3	492.7	99.3	156.0
DCVC-HEM	23.8	10.5	27.5	33.4	11.3	139.1	24.2	38.5
DCVC-DC	-7.8	-10.0	-0.9	6.1	-15.0	33.3	9.4	2.2
DCVC-SDD	-3.5	-3.8	12.2	13.5	-15.7	160.0	8.3	24.4
DCVC-SDD_f	-16.7	-11.7	-9.0	-3.2	-18.8	-20.1	-4.3	-12.0
DCVC-FM	-19.8	-10.9	-12.2	-12.9	-27.1	-27.7	16.3	-13.5
DCMVC	-25.2	-20.9	-17.8	-18.0	-31.4	-23.5	-6.3	-20.4
Ours	-37.7	-29.5	-32.5	-30.4	-41.1	-31.6	-17.4	-31.5

DCVC-SDD [33] is our small baseline model.

DCVC-SDD_f denotes DCVC-SDD trained with 32-frame cascaded training.

The quality indexes of DCVC-FM are set as 36, 45, 54, 63 to match the bit-rate range of DCVC-DC.

TABLE X
BD-RATE (%) COMPARISON FOR YUV-PSNR. 96 FRAMES WITH INTRA-PERIOD = 32. THE ANCHOR IS VTM.

	UVG	MCL-JCV	HEVC B	HEVC C	HEVC D	HEVC E	USTC-TD	Average
VTM	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
ECM	-15.5	-18.3	-19.5	-21.5	-20.1	-18.8	-17.2	-18.7
DCVC-DC_y	-17.2	-11.0	-11.6	-13.1	-28.8	-18.1	8.2	-13.1
DCVC-DC	-14.7	-9.8	-5.6	-11.6	-30.9	-20.5	10.3	-11.8
DCVC-SDD	-7.0	-0.3	-4.2	-2.3	-25.7	-10.6	9.6	-5.8
DCVC-FM	-25.0	-15.6	-16.6	-17.7	-33.4	-29.9	8.6	-18.5
DCMVC	-18.5	-10.3	-2.3	-8.4	-28.4	-21.1	1.8	-12.5
NVC-1B	-17.7	-15.8	-17.6	-19.7	-34.9	-23.5	-10.0	-19.9

DCVC-SDD is our small baseline model.

DCVC-DC_y denotes its publicly available model re-optimized for YUV.

ECM-13.0 only supports encoding YUV420 videos currently.

- -c {config file name} -InputFile={input file name}
 -InputChromaFormat={input chroma format} -
 FrameRate={frame rate} -DecodingRefreshType=2
 -InputBitDepth=8 -FramesToBeEncoded=96 -
 SourceWidth={width} -SourceHeight={height}
 -IntraPeriod=32 -QP={qp} -Level=6.2 -
 BitstreamFile={bitstream file name}

For neural video coding models, we choose DCVC [72], DCVC-TCM [31], DCVC-HEM [35], DCVC-DC [35], DCVC-FM [46], DCMVC [42], and our small baseline model—DCVC-SDD [33] as our benchmarks. For fair comparison, we further

fine-tune DCVC-SDD with 32-frame cascaded training, denoted as DCVC-SDD_f.

4) *Evaluation Metrics*: For evaluation, we use RGB-PSNR as the distortion metric when testing the video sequences. When testing YUV420 videos, following DCVC-DC [35], we use the compound YUV PSNR as the distortion evaluation metric. The weight of YUV components is set to 6:1:1 [106]. Bits per pixel (bpp) is used as the bitrate evaluation metric. BD-rate [107] is used to compare the compression performance of different models, where negative numbers indicate bitrate saving and positive numbers indicate bitrate increasing.

B. Experimental Results

1) *Objective Comparison Results*: When testing RGB videos under intra-period 32 for 96 frames, we illustrate the rate-distortion curves on the HEVC, USTC-TD, UVG, and MCL-JCV RGB video datasets in Fig. 2. The curves show that our proposed large neural video coding model—NVC-1B achieves the best compression performance among all compared codecs. We list the detailed BD-rate comparison results in Table VIII. The results show that our NVC-1B achieves an average 27.8% BD-rate reduction over VTM-13.2 across all test datasets, which is much better than that of DCVC-SDD_f (15.5%). It also surpasses the SOTA neural video coding models—DCVC-FM (9.9%) and DCMVC (19.4%).

When evaluating with an intra-period of -1 for all frames, we present the rate-distortion curves in Fig. 3 and report the corresponding BD-rate results in Table IX. For fair comparison, under the all-frame testing setting with an intra-period of -1, we evaluate DCVC-SDD, DCVC-SDD_f, and our model with periodic refresh in DCVC-FM [46] to alleviate error accumulation over long sequences. The results show that, after long-sequence fine-tuning, DCVC-SDD_f achieves a substantial improvement over the original DCVC-SDD, improving from a 24.4% average loss to a 12.0% average gain. Compared with this stronger baseline, NVC-1B still achieves a 19.5% average BD-rate gain over DCVC-SDD_f. Furthermore, it also outperforms the current SOTA neural video coding models—DCVC-FM (−13.5%) and DCMVC (−20.4%).

When evaluating YUV format, we report the BD-rate results in Table X. For NVC-1B, we adopt a format-transformation-based evaluation: the input YUV sequence is converted to RGB, compressed by the RGB-optimized model, and then converted back to YUV for evaluation. This setting avoids the substantial additional cost of re-finetuning the large model in the YUV domain and also provides a fairer comparison under a single-model evaluation protocol. Accordingly, DCVC-DC, DCVC-SDD, and DCMVC are also evaluated using their released RGB-optimized models, while DCVC-DC_y denotes the publicly available YUV-optimized model. The results indicate that such a format transformation still causes a noticeable performance loss. Under this practical YUV420 evaluation setting, our method still achieves a 19.9% average BD-rate reduction over VTM, which is the best average result among the compared methods, including DCVC-FM and ECM. We also observe a performance trade-off when a single model is used for multiple color spaces: the average gain over DCVC-FM drops from 17.9% in RGB to 1.4% in YUV.

2) *Subjective Comparison Results*: We visualize the reconstructed frames of our proposed NVC-1B and its small baseline model—DCVC-SDD in Fig. 4. By comparing their subjective qualities, we observe that the frames reconstructed by NVC-1B can retain more textures with a lower bitrate. We take the *BQTerrace* sequence of the HEVC Class B dataset, the *videoSRC24* sequence of the MCL-JCV dataset, and the *Johnny* sequence of the HEVC Class E dataset as examples. For the *BQTerrace*, our NVC-1B model achieves a reconstructed frame of 32.46 dB at a similar bitrate of 0.019 bpp (31.60 dB for DCVC-SDD).

TABLE XI
AVERAGE ENCODING/DECODING TIME FOR A 1080P FRAME (IN SECONDS)

Schemes	Enc Time	Dec Time
VTM	743.88 s	0.31 s
ECM	5793.88 s	1.28 s
DCVC-SDD	0.89 s	0.70 s
Ours	4.44 s	3.54 s

When observing the grass in *BQTerrace*, it is evident that our NVC-1B model preserves more fine details. For the *videoSRC24* sequence, our NVC-1B model can use the bitrate of 0.016 bpp (0.017 bpp for DCVC-SDD) to achieve a reconstructed frame of 38.53 dB (36.81 dB for DCVC-SDD). Comparing the inner edge textures of the tire, our NVC-1B can successfully reconstruct these textures, while DCVC-SDD cannot. For the *Johnny* sequence, our NVC-1B model achieves a reconstructed frame of 37.69 dB at a similar bitrate of 0.004 bpp, compared to 35.48 dB for DCVC-SDD. Zooming in on the eyes of the male host, our model delivers a significantly clearer reconstruction.

3) *Complexity and Encoding/Decoding Time*: The complexity of NVC-1B is 88,772 kMAC/ pixel, which is 48 times that of our small baseline model (1848 kMAC/ pixel). The number of parameters of NVC-1B is 1355.95 M (1.36B), which is 62.5 times that of our small baseline model (21.7 M).

We also compare the encoding and decoding time, including the time for model inference, entropy modeling, entropy coding, and data transfer between CPU and GPU. All learned video codecs are run on an NVIDIA A800 GPU. As shown in Table XI, our NVC-1B model takes 4.44 s and 3.54 s per 1080p frame for encoding and decoding, respectively, about five times longer than our small baseline model (0.89 s and 0.70 s). With the fast development of computing power of hardware and on-device deployment of large models [51], [61], [62], [63], [64], [65], [66], [67], [68], [69], we believe that the on-device deployment of our NVC-1B model is feasible in the future.

C. Analysis

1) *Ablation Study on Scaling Laws With Model Parameters*: To investigate the scaling law in large neural video compression, we conduct joint end-to-end scaling-up experiments. These experiments involved jointly scaling all modules that previously demonstrated stable per-module gains and training them sufficiently. We provide three progressively scaled models with 21 M (Small), 212 M (Medium), and 1.36B (Large) parameters, where the small model corresponds to our baseline model with full 32-frame cascaded finetune, DCVC-SDD_f. As shown in Table XII, the experimental results indicate that the medium-sized model achieves an average bitrate saving of 6.2% across multiple datasets compared to the small model. The large model, on the other hand, achieves an average bitrate saving of 15.1%. Furthermore, unlike the scaling law observed in neural image compression [47], scaling up model parameters in neural video compression results in greater performance gains. This demonstrates the significant potential of increasing the parameter size in neural video compression models, which

TABLE XII
ABLATION STUDY ON MODEL SCALING LAWS. BD-RATE (%) COMPARISON
FOR RGB-PSNR. THE EVALUATION USES 96 FRAMES WITH INTRA-PERIOD =
32. THE ANCHOR IS THE SMALL MODEL DCVC-SDD.

	UVG	MCL- JCV	B	C	D	E	USTC- TD	Avg
Small (21M)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Medium (212M)	-6.4	-4.9	-7.4	-11.8	-6.3	-4.8	-1.8	-6.2
Large (1.36B)	-14.7	-16.2	-16.7	-19.9	-17.2	-11.3	-9.8	-15.1

involve more complex module structures and training strategies, thereby highlighting broader potential for improvement in video compression.

2) *Analysis of Transform Energy Compaction*: To explore why our proposed large video coding model—NVC-1B can bring performance gain, we analyze the transform energy compaction. We calculate the average bitrate ratio of each channel of the contextual latent representation $\hat{y}_t$ over the HEVC Class C and D datasets. Specifically, the contextual bitrate ratio (CR) of channel c for each video sequence is first calculated as (7).

$$CR = \frac{\sum_t \{-\log_2(p(\hat{y}_t^c))\}}{\sum_c \sum_t \{-\log_2(p(\hat{y}_t^c))\}}. \quad (7)$$

Then, we calculate the average contextual bitrate ratio over all sequences. We sort the average contextual bitrate ratio of each channel from largest to smallest to analyze the transform energy compaction. As illustrated in Fig. 5, for contextual bitrate ratio, we present the top 100 channels with obvious bitrate overhead. Comparing the channel bitrate distribution, we find that the contextual latent representations generated by our proposed large video coding model have higher transform energy compaction than our small baseline model—DCVC-SDD. More bitrates are concentrated in fewer channels.

In addition to the contextual latent representations, we also calculate the channel bitrate ratio of motion latent representations $\hat{m}_t$. Similarly, the motion bitrate ratio (MR) of channel c for each video sequence is first calculated as (8).

$$MR = \frac{\sum_t \{-\log_2(p(\hat{m}_t^c))\}}{\sum_c \sum_t \{-\log_2(p(\hat{m}_t^c))\}}. \quad (8)$$

Then, we calculate the average motion bitrate ratio over all sequences. As presented in Fig. 5, by ranking the average motion bitrate ratio of each channel, we find that although we allocate most of the parameters to contextual parts, the transform energy compaction can be improved for motion compression by end-to-end training. The motion channel bitrate distribution of our large model is also more concentrated. The analysis indicates that our large video coding model can obtain a higher compression performance by increasing the transform energy compaction. More signal energy is concentrated in fewer channels, which is beneficial for entropy coding.

3) *Analysis of Temporal Context Mining Accuracy*: We further analyze the temporal context mining accuracy to explore why our proposed large video coding model—NVC-1B can bring performance gain. We visualize the temporal context C_t^0

predicted by NVC-1B and its small baseline model—DCVC-SDD in Fig. 6. By zooming in the temporal contexts, we find that the context predicted by NVC-1B has smoother object edges, whereas that predicted by DCVC-SDD has obvious prediction artifacts. For example, the edges of the horse's tail and saddle predicted by DCVC-SDD have much noise, which seriously reduces the temporal prediction accuracy. The analysis indicates that our NVC-1B model can obtain more accurate temporal predictions, thus achieving higher compression performance.

4) *Analysis of Scaling up for Motion Encoder-Decoder*: In Section IV-A1, we observe a strange phenomenon that if we slightly increase the model size of the motion encoder and decoder, it can lead to performance gains. However, the performance gain will decrease if we continuously increase the model size. To explore the reason for this strange phenomenon, we visualize the warp frames predicted by the small baseline model—DCVC-SDD, M_1 , and M_3 models mentioned in Section IV-A1. As illustrated in Fig. 7, we find that, at a similar bitrate, the M_1 model with 52M parameters can obtain a more accurate warp frame than DCVC-SDD with 21M parameters. The arm and bubbles predicted by the M_1 model have fewer artifacts. However, the accuracy of the warp frame predicted by M_3 decreases dramatically. Its prediction artifacts of arms and bubbles increase significantly. The analysis indicates that the model size of the motion encoder-decoder is not always the bigger the better. An inappropriate model size will reduce the accuracy of temporal prediction.

5) *Analysis of Feature Spatial Correlation for Different Model Architecture*: To further understand the influence of model architecture, we conduct a spatial correlation analysis of the normalized contextual latent representation $(y_t - \mu_y)/\sigma_y$ for the CNN model M_{18} , the mixed CNN-Transformer model M_{21} , and the Transformer-only model M_{24} , following the analysis style of [108]. Specifically, the spatial correlation is first computed over relative offsets in the range $[-10, 10]$ and then aggregated into a 7×7 map by 3×3 block averaging for better visualization, where the displayed coordinates correspond to block-averaged offsets in $[-3, 3]$. As shown in Fig. 8, the CNN model produces a more compact spatial correlation pattern, indicating that its latent representation is more effectively decorrelated. This observation is consistent with the RD results in Tables IV and V, where the CNN model also achieves better coding performance. These results suggest that, under the current video coding framework and training setting, CNN-based scaling is more stable and effective than the tested Transformer variants. This observation is also broadly consistent with ConvNeXt [109], which shows that carefully designed modern CNN architectures can remain highly competitive with Transformer models in vision tasks. However, we note that this does not imply CNN is universally superior to Transformer. The results indicate that identifying a more suitable Transformer architecture and its effective scaling law for neural video compression remains an open problem.

VI. CONCLUSION

In this paper, we scale up the model sizes of different parts of a neural video coding model to analyze the influence of

model size on compression performance. In addition, we use different architectures to implement a neural video coding model to analyze the influence of model architecture on compression performance. Based on our exploration, we design the first large neural video coding model—NVC-1B. Experimental results show that our large model achieves a significant video compression performance improvement over state-of-the-art neural video compression models.

Despite the great quantitative and qualitative improvements of our NVC-1B, some limitations remain in our first exploration process. First, due to the limitation of accessing open-source training codes of other neural video coding models, we can only conduct large model exploration experiments on a state-of-the-art baseline with DCVC-DC architecture. Some exploration conclusions may not be appropriate for other neural video coding models. Second, while NVC-1B shows promising performance gains, there remains potential for further improvement. The current partial cascaded fine-tuning strategy cannot match the performance achieved by the full cascaded fine-tuning approach without group separation. However, given present hardware configurations and GPU memory management constraints, training such large-scale models end-to-end poses substantial implementation challenges. In view of these limitations, we will continue to pursue more efficient model architectures and more optimized training strategies to further improve performance in future work.

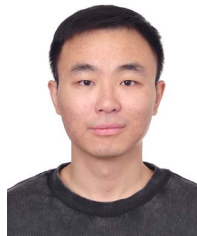
Our results suggest that large models may play an important role in future video coding development and provide useful insights for improving coding efficiency.

REFERENCES

- [1] T. Wiegand, G. J. Sullivan, G. Bjontegaard, and A. Luthra, "Overview of the H.264/AVC video coding standard," *IEEE Trans. Circuits Systems Video Technol.*, vol. 13, no. 7, pp. 560–576, Jul. 2003.
- [2] G. J. Sullivan, J.-R. Ohm, W.-J. Han, and T. Wiegand, "Overview of the high efficiency video coding (HEVC) standard," *IEEE Trans. Circuits Systems Video Technol.*, vol. 22, no. 12, pp. 1649–1668, Dec. 2012.
- [3] B. Bross et al., "Overview of the versatile video coding (VVC) standard and its applications," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 10, pp. 3736–3764, Oct. 2021.
- [4] A. Habibiyan, T. v. Rozendaal, J. M. Tomczak, and T. S. Cohen, "Video compression with rate-distortion autoencoders," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2019, pp. 7032–7041.
- [5] W. Sun, C. Tang, W. Li, Z. Yuan, H. Yang, and Y. Liu, "High-quality single-model deep video compression with frame-conv3D and multi-frame differential modulation," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 239–254.
- [6] J. Liu et al., "Conditional entropy coding for efficient video compression," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 453–468.
- [7] F. Mentzer et al., "VCT: A video compression transformer," in *Adv. Neural Inf. Process. Syst.*, 2022, pp. 13091–13103.
- [8] H. Chen, B. He, H. Wang, Y. Ren, S. N. Lim, and A. Shrivastava, "NeRV: Neural representations for videos," in *Adv. Neural Inf. Process. Syst.*, 2021, pp. 21557–21568.
- [9] H. Chen, M. Gwilliam, S.-N. Lim, and A. Shrivastava, "HNeRV: A hybrid neural representation for videos," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 10270–10279.
- [10] Q. Zhao, M. S. Asif, and Z. Ma, "DNeRV: Modeling inherent dynamics via difference neural representation for videos," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 2031–2040.
- [11] H. M. Kwan, G. Gao, F. Zhang, A. Gower, and D. Bull, "HiNeRV: Video compression with hierarchical encoding-based neural representation," in *Adv. Neural Inf. Process. Syst.*, 2024, pp. 72692–72704.
- [12] H. M. Kwan, G. Gao, F. Zhang, A. Gower, and D. Bull, "NVRC: Neural video representation compression," in *Adv. Neural Inf. Process. Syst.*, 2024, pp. 132440–132462.
- [13] H. Liu, H. Shen, L. Huang, M. Lu, T. Chen, and Z. Ma, "Learned video compression via joint spatial-temporal correlation exploration," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 11580–11587.
- [14] O. Rippel, A. G. Anderson, K. Tatwawadi, S. Nair, C. Lytle, and L. Bourdev, "ELF-VC: Efficient learned flexible-rate video coding," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2021, pp. 14479–14488.
- [15] G. Lu, X. Zhang, W. Ouyang, L. Chen, Z. Gao, and D. Xu, "An end-to-end learning framework for video compression," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 10, pp. 3292–3308, Oct. 2021.
- [16] J. Lin, D. Liu, H. Li, and F. Wu, "M-LVC: Multiple frames prediction for learned video compression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 3546–3554.
- [17] Z. Hu, G. Lu, and D. Xu, "FVC: A new framework towards deep video compression in feature space," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 1502–1511.
- [18] E. Agustsson, D. Minnen, N. Johnston, J. Balle, S. J. Hwang, and G. Toderici, "Scale-space flow for end-to-end optimized video compression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 8503–8512.
- [19] Z. Cheng, H. Sun, M. Takeuchi, and J. Katto, "Learning image and video compression through spatial-temporal energy compaction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 10071–10080.
- [20] O. Rippel, S. Nair, C. Lew, S. Branson, A. G. Anderson, and L. Bourdev, "Learned video compression," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 3454–3463.
- [21] A. Djelouah, J. Campos, S. Schaub-Meyer, and C. Schroers, "Neural inter-frame compression for video coding," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 6421–6429.
- [22] R. Yang, F. Mentzer, L. Van Gool, and R. Timofte, "Learning for video compression with recurrent auto-encoder and recurrent probability model," *IEEE J. Sel. Topics Signal Process.*, vol. 15, no. 2, pp. 388–401, Feb. 2021.
- [23] B. Liu, Y. Chen, S. Liu, and H.-S. Kim, "Deep learning in latent space for video prediction and compression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 701–710.
- [24] H. Liu et al., "Neural video coding using multiscale motion compensation and spatiotemporal context model," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 8, pp. 3182–3196, Aug. 2021.
- [25] M. A. Yilmaz and A. M. Tekalp, "End-to-end rate-distortion optimized learned hierarchical bi-directional video compression," *IEEE Trans. Image Process.*, vol. 31, pp. 974–983, 2021.
- [26] Z. Chen, T. He, X. Jin, and F. Wu, "Learning for video compression," *IEEE Trans. Circuits Systems Video Technol.*, vol. 30, no. 2, pp. 566–576, Feb. 2020.
- [27] K. Lin, C. Jia, X. Zhang, S. Wang, S. Ma, and W. Gao, "DMVC: Decomposed motion modeling for learned video compression," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 33, no. 7, pp. 3502–3515, Jul. 2023.
- [28] Z. Guo, R. Feng, Z. Zhang, X. Jin, and Z. Chen, "Learning cross-scale weighted prediction for efficient neural video compression," *IEEE Trans. Image Process.*, vol. 32, pp. 3567–3579, 2023.
- [29] H. Guo, S. Kwong, C. Jia, and S. Wang, "Enhanced motion compensation for deep video compression," *IEEE Signal Process. Lett.*, vol. 30, pp. 673–677, 2023.
- [30] R. Yang, R. Timofte, and L. V. Gool, "Advancing learned video compression with in-loop frame prediction," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 33, no. 5, pp. 2410–2423, May 2023.
- [31] X. Sheng, J. Li, B. Li, L. Li, D. Liu, and Y. Lu, "Temporal context mining for learned video compression," *IEEE Trans. Multimedia*, vol. 25, pp. 7311–7322, 2022.
- [32] F. Wang, H. Ruan, F. Xiong, J. Yang, L. Li, and R. Wang, "Butterfly: Multiple reference frames feature propagation mechanism for neural video compression," in *Proc. Data Compression Conf.*, 2023, pp. 198–207.
- [33] X. Sheng, L. Li, D. Liu, and H. Li, "Spatial decomposition and temporal fusion based inter prediction for learned video compression," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 34, no. 7, pp. 6460–6473, Jul. 2024.
- [34] X. Sheng, L. Li, and D. Liu, "VNVC: A versatile neural video coding framework for efficient human-machine vision," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 7, pp. 4579–4596, Jul. 2024.

- [35] J. Li, B. Li, and Y. Lu, "Hybrid spatial-temporal entropy modelling for neural video compression," in *Proc. 30th ACM Int. Conf. Multimedia*, 2022, pp. 1503–1511.
- [36] Y.-H. Ho, C.-P. Chang, P.-Y. Chen, A. Gnutti, and W.-H. Peng, "CANFVC: Conditional augmented normalizing flows for video compression," in *Proc. 17th Eur. Conf. Comput. Vis.*, 2022, pp. 207–223.
- [37] D. Jin, J. Lei, B. Peng, Z. Pan, L. Li, and N. Ling, "Learned video compression with efficient temporal context learning," *IEEE Trans. Image Process.*, vol. 32, pp. 3188–3198, 2023.
- [38] R. Lin, M. Wang, P. Zhang, S. Wang, and S. Kwong, "Multiple hypotheses based motion compensation for learned video compression," *Neurocomputing*, vol. 548, 2023, Art. no. 126396.
- [39] J. Li, B. Li, and Y. Lu, "Neural video compression with diverse contexts," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 22616–22626.
- [40] Y. Bian, X. Sheng, L. Li, and D. Liu, "LSSVC: A learned spatially scalable video coding scheme," *IEEE Trans. Image Process.*, vol. 33, pp. 3314–3327, 2024.
- [41] C. Tang, X. Sheng, Z. Li, H. Zhang, L. Li, and D. Liu, "Offline and online optical flow enhancement for deep video compression," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 5118–5126.
- [42] C. Tang, Z. Li, Y. Bian, L. Li, and D. Liu, "Neural video compression with context modulation," in *Proc. Comput. Vis. Pattern Recognit. Conf.*, 2025, pp. 12553–12563.
- [43] W. Jiang, J. Li, K. Zhang, and L. Zhang, "ECVC: Exploiting non-local correlations in multiple frames for contextual video compression," in *Proc. Comput. Vis. Pattern Recognit. Conf.*, 2025, pp. 7331–7341.
- [44] Z. Jia et al., "Towards practical real-time neural video compression," in *Proc. Comput. Vis. Pattern Recognit. Conf.*, 2025, pp. 12543–12552.
- [45] Y. Bian, C. Tang, L. Li, and D. Liu, "Augmented deep contexts for spatially embedded video coding," in *Proc. Comput. Vis. Pattern Recognit. Conf.*, 2025, pp. 2094–2104.
- [46] J. Li, B. Li, and Y. Lu, "Neural video compression with feature modulation," in *Proc. Comput. Vis. Pattern Recognit. Conf.*, 2024, pp. 26099–26108.
- [47] Y. Li, H. Zhang, L. Li, D. Liu, and F. Wu, "Scaling learned image compression models up to 1 billion," 2025, *arXiv:2508.09075*.
- [48] J. Kaplan et al., "Scaling laws for neural language models," 2020, *arXiv:2001.08361*.
- [49] T. Brown et al., "Language models are few-shot learners," in *Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.
- [50] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
- [51] A. Liu et al., "Deepseek-V3 technical report," 2024, *arXiv:2412.19437*.
- [52] A. Kirillov et al., "Segment anything," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 4015–4026.
- [53] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, "Depth anything: Unleashing the power of large-scale unlabeled data," 2024, *arXiv:2401.10891*.
- [54] A. Dosovitskiy et al., "An image is worth 16 × 16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.
- [55] M. Dehghani et al., "Scaling vision transformers to 22 billion parameters," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 7480–7512.
- [56] S. Batifol et al., "FLUX. 1 Kontext: Flow matching for in-context image generation and editing in latent space," 2025, *arXiv:2506.15742*.
- [57] C. Wu et al., "Qwen-image technical report," 2025, *arXiv:2508.02324*.
- [58] T. Wan et al., "WAN: Open and advanced large-scale video generative models," 2025, *arXiv:2503.20314*.
- [59] Z. Liu et al., "Swin transformer V2: Scaling up capacity and resolution," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 12009–12019.
- [60] "VTM-13.2," Accessed: Mar. 2, 2022. [Online]. Available: https://vcgit.hhi.fraunhofer.de/jvet/VVCSsoftware_VTM/
- [61] J. Lin et al., "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proc. Mach. Learn. Syst.*, vol. 6, pp. 87–100, 2024.
- [62] S. Ma et al., "The era of 1-bit LLMs: All large language models are in 1.58 bits," 2024, *arXiv:2402.17764*.
- [63] J. Kaddour, J. Harris, M. Mozes, H. Bradley, R. Raileanu, and R. McHardy, "Challenges and applications of large language models," 2023, *arXiv:2307.10169*.
- [64] Y. Gu, L. Dong, F. Wei, and M. Huang, "MiniLLM: Knowledge distillation of large language models," 2023, *arXiv:2306.08543*.
- [65] Z. Yao, X. Wu, C. Li, S. Youn, and Y. He, "Exploring post-training quantization in LLMs from comprehensive study to low rank compensation," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 19377–19385.
- [66] G. Team et al., "Gemini: A family of highly capable multimodal models," 2023, *arXiv:2312.11805*.
- [67] M. Abdin et al., "Phi-3 technical report: A highly capable language model locally on your phone," 2024, *arXiv:2404.14219*.
- [68] S. Mehta et al., "OpenELM: An efficient language model family with open training and inference framework," 2024, *arXiv:2404.14619*.
- [69] J. Xu et al., "On-device language models: A comprehensive review," 2024, *arXiv:2409.00088*.
- [70] C.-Y. Wu, N. Singhal, and P. Krahenbuhl, "Video compression through image interpolation," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 416–431.
- [71] G. Lu, W. Ouyang, D. Xu, X. Zhang, C. Cai, and Z. Gao, "DVC: An end-to-end deep video compression framework," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Long Beach, CA, USA, 2019, pp. 11006–11015.
- [72] J. Li, B. Li, and Y. Lu, "Deep contextual video compression," in *Adv. Neural Inf. Process. Syst.*, 2021, pp. 18114–18125.
- [73] Z. Hu, G. Lu, J. Guo, S. Liu, W. Jiang, and D. Xu, "Coarse-to-fine deep video coding with hyperprior-guided mode prediction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 5921–5930.
- [74] H. Liu, M. Lu, Z. Chen, X. Cao, Z. Ma, and Y. Wang, "End-to-end neural video coding using a compound spatiotemporal representation," *IEEE Trans. Circuits Systems Video Technol.*, vol. 32, no. 8, pp. 5650–5662, Aug. 2022.
- [75] Y. Shi, Y. Ge, J. Wang, and J. Mao, "AlphaVC: High-performance and efficient learned video compression," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 616–631.
- [76] J. Xiang, K. Tian, and J. Zhang, "MIMT: Masked image modeling transformer for video compression," in *Proc. 11th Int. Conf. Learn. Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=j9m-mVnndbm>
- [77] T. Xu et al., "Bit allocation using optimization," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 38377–38399.
- [78] Y.-H. Chen et al., "MaskCRT: Masked conditional residual transformer for learned video compression," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 34, no. 11, pp. 11980–11992, Nov. 2024.
- [79] H. Kim, M. Bauer, L. Theis, J. R. Schwarz, and E. Dupont, "C3: High-performance and low-complexity neural compression from a single image or video," 2023, *arXiv:2312.02753*.
- [80] M. Lu, Z. Duan, F. Zhu, and Z. Ma, "Deep hierarchical video compression," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 8859–8867.
- [81] P. Du, Y. Liu, and N. Ling, "CGVC-T: Contextual generative video compression with transformers," *IEEE Trans. Emerg. Sel. Topics Circuits Syst.*, vol. 14, no. 2, pp. 209–223, Jun. 2024.
- [82] W. Jiang, J. Li, K. Zhang, and L. Zhang, "ECVC: Exploiting non-local correlations in multiple frames for contextual video compression," 2024, *arXiv:2410.09706*.
- [83] J. Dai et al., "Deformable convolutional networks," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 764–773.
- [84] N. Du et al., "GLaM: Efficient scaling of language models with mixture-of-experts," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 5547–5569.
- [85] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," *J. Mach. Learn. Res.*, vol. 24, no. 240, pp. 1–113, 2023.
- [86] R. Anil et al., "PaLM 2 technical report," 2023, *arXiv:2305.10403*.
- [87] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
- [88] A. Meta, "Introducing meta LLaMA 3: The most capable openly available LLM to date," *Meta AI*, 2024.
- [89] Mistral AI, "Introducing mistral 7B," 2023, Accessed: May 5, 2025. [Online]. Available: <https://mistral.ai/news/announcing-mistral-7b/>
- [90] X. Chen et al., "PaLI: A jointly-scaled multilingual language-image model," 2022, *arXiv:2209.06794*.
- [91] M. Cherti et al., "Reproducible scaling laws for contrastive language-image learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 2818–2829.
- [92] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8748–8763.
- [93] A. Ranjan and M. J. Black, "Optical flow estimation using a spatial pyramid network," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 4161–4170.
- [94] J. Ballé, V. Laparra, and E. P. Simoncelli, "End-to-end optimized image compression," in *Proc. Int. Conf. Learn. Representations*, Toulon, France, 2017. [Online]. Available: <https://arxiv.org/abs/1611.01704>
- [95] J. Ballé, D. Minnen, S. Singh, S. J. Hwang, and N. Johnston, "Variational image compression with a scale hyperprior," in *Proc. 6th Int. Conf. Learn. Representations*, Vancouver, BC, Canada, 2018. [Online]. Available: <https://openreview.net/forum?id=rkcQFMZRB>
- [96] J. Liu, H. Sun, and J. Katto, "Learned image compression with mixed transformer-CNN architectures," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 14388–14397.

- [97] H. Guo, S. Kwong, D. Ye, and S. Wang, "Enhanced context mining and filtering for learned video compression," *IEEE Trans. Multimedia*, vol. 26, pp. 3814–3826, 2023.
- [98] Z. Chen et al., "Neural video compression with spatio-temporal cross-covariance transformers," in *Proc. 31st ACM Int. Conf. Multimedia*, 2023, pp. 8543–8551.
- [99] L. Qi, J. Li, B. Li, H. Li, and Y. Lu, "Motion information propagation for neural video compression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 6111–6120.
- [100] F. Bossen, "Common HM test conditions and software reference configurations (JCTVC-L1100)," *Joint Collaborative Team Video Coding ITU-T SG*, 2013.
- [101] Z. Li et al., "USTC-TD: A test dataset and benchmark for image and video coding in 2020s," *IEEE Trans. Multimedia*, vol. 28, pp. 269–284, 2025.
- [102] A. Mercat, M. Viitanen, and J. Vanne, "UVG dataset: 50/120fps 4K sequences for video codec analysis and development," in *Proc. 11th ACM Multimedia Syst. Conf.*, 2020, pp. 297–302.
- [103] H. Wang et al., "MCL-JCV: A JND-based H.264/AVC video quality assessment dataset," in *Proc. IEEE Int. Conf. Image Process.*, 2016, pp. 1509–1513.
- [104] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [105] "ECM-13.0," Accessed: Jul. 4, 2024. [Online]. Available: <https://vcgit.hhi.fraunhofer.de/ecm/ECM>
- [106] J.-R. Ohm, G. J. Sullivan, H. Schwarz, T. K. Tan, and T. Wiegand, "Comparison of the coding efficiency of video coding standards—including high efficiency video coding (HEVC)," *IEEE Trans. Circuits Systems Video Technol.*, vol. 22, no. 12, pp. 1669–1684, Dec. 2012.
- [107] G. Bjontegaard, "Calculation of average PSNR differences between RD-curves," *VCEG-M33*, 2001.
- [108] Y. Zhu, Y. Yang, and T. Cohen, "Transformer-based transform coding," in *Proc. Int. Conf. Learn. Representations*, 2022.
- [109] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A convnet for the 2020s," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 11976–11986.



Chuanbo Tang received the BS degree in electronic information engineering from the Hefei University of Technology, Hefei, China, in 2021. He is currently working toward the PhD degree with the Department of Electronic Engineering and Information Science, University of Science and Technology of China, Hefei, China. His research interests include image/video coding, signal processing, and machine learning.



Xihua Sheng (Member, IEEE) received the BS degree in automation from Northeastern University, Shenyang, China, in 2019, and the PhD degree in electronic engineering from the University of Science and Technology of China (USTC), Hefei, Anhui, China, in 2024. He is currently a postdoctoral fellow in computer science from the City University of Hong Kong (CityU). His research interests include image/video/point cloud coding, signal processing, and machine learning.



100 papers in international journals and conferences. He has more than 20 granted patents. He has several technique proposals adopted by standardization groups. He received the Multimedia Rising Star 2023. He received the Best 10% Paper Award at the 2016 IEEE Visual Communications and Image Processing (VCIP) and the 2019 IEEE International Conference on Image Processing (ICIP). He serves as a senior associate editor for *IEEE Transactions on Circuits and Systems for Video Technology* since 2026 and an associate editor for *IEEE Transactions on Multimedia* since 2025.



Dong Liu (Senior Member, IEEE) received the BS and Ph.D. degrees in electrical engineering from the University of Science and Technology of China (USTC), Hefei, China, in 2004 and 2009, respectively. He was a member of research staff with Nokia Research Center, Beijing, China, from 2009 to 2012. He has been a faculty member with USTC since 2012 and currently holds the position of full professor. His research interests include image and video compression/coding and visual intelligence. He has authored or co-authored more than 300 papers in international journals and conferences. He has more than 40 granted patents, and dozens of technical proposals adopted by standardization groups. He received 2009 IEEE TCSVT Best Paper Award. He is an elected member of IVMSP-TC of IEEE SP Society and an elected member of MSA-TC of IEEE CAS Society. He serves or had served as the Chair of IEEE 1857.11 Standard Sub-Working-Group, which received the IEEE SA Emerging Technology Award in 2025. He also serves as an associate editor for *IEEE Transactions on Image Processing*, a Guest Editor for *IEEE Transactions on Circuits and Systems for Video Technology*, and a TPC co-chair for ICME 2026.



Feng Wu (Fellow, IEEE) received the BS degree in electrical engineering from Xidian University, in 1992, and the MS and PhD degrees in computer science from the Harbin Institute of Technology in 1996 and 1999, respectively. He is a professor and vice president with the University of Science and Technology of China, Hefei, China. Previously, he was a principle researcher and research manager with Microsoft Research Asia, Beijing, China. His research interests include various aspects of multimedia technology and artificial intelligence. He has authored or co-authored more than 150 journal papers (including several dozens of IEEE Transactions papers) and top conference papers on MOBICOM, SIGIR, CVPR, and ACM MM. He has more than 100 granted US patents. His 15 techniques have been adopted into international video coding standards. He had served as the editor-in-chief for *IEEE Transactions on Circuits and System for Video Technology*, an associate editor for *IEEE Transactions on Image Processing* and *IEEE Transactions on Multimedia*. He also served as general chair in ICME 2019, TPC Chair in MMSP 2011, VCIP 2010 and PCM 2009. He received the Mac Van Valkenburg award from IEEE CAS Society in 2021. He received the best paper awards in IEEE TCSVT 2009, VCIP 2016, PCM 2008, and VCIP 2007, and Best associate editor Award of *IEEE Transactions on Image Processing* in 2018. He is a member of the Chinese Academy of Engineering.